Reproducing HLS-Eval and Analog System Benchmarks from Cong Hao's Group

Results, what broke, and a proposed first project

Michael Incorvaia

August 14, 2026

Abstract

This note reports two reproductions of work co-authored by Cong (Callie) Hao, intended as a concrete reading of the lab rather than a survey. First, I reproduced the zero-shot HLS generation baseline from *HLS-Eval* (Abi-Karam & Hao, ICLAD 2025) on 57 designs without Vitis HLS, then re-ran the same harness on three post-paper models. DeepSeek-V3 lands at 46.3% run@1 versus the paper's 63.3%, but 63.2% versus 65.9% at run@5; the pass@1 gap is largely a fixed `max_tokens=8192` cap. Under that same cap, Qwen3-Coder reaches 67.4% run@1. Second, I treated Hasler & Hao's TRETS 2024 analog benchmark paper and the Ige/Hasler/Hao JLPEA 2023 analog-HLS paper as one project: re-derived the Fig. 3/11 scaling anchors and the Fig. 17–18 Case II CAB arithmetic (128 WTA CABs / 98 SoC CABs = 1.31 chips — the "barely more than one SoC FPAA, fits in two" claim holds), and executed TRETS Fig. 16's acoustic program as a digital macromodel. On a 4-word Speech Commands subset the VMM+WTA classifier reaches 70% held-out accuracy (chance 25%). The AnalogBenchmarks artifact URL cited in TRETS footnote 1 was already 404 on the Wayback Machine in November 2023. I did not reproduce SmartPAF or CRYPTONITE; both are discussed as future work, as is AMS-Eval — an HLS-Eval-style LLM harness on the Fig. 15 analog block library, which is the project I would want to start in the group.

Contents

1 What this note is **2**

2 HLS-Eval: zero-shot generation, one year later **2**
2.1 What was reproduced . . . 2
2.2 Paper vs. this run . . . 2
2.3 Extension: models that postdate the paper . . . 3
2.4 What broke, independent of any model . . . 3

3 Improvements I considered and did not run **4**
3.1 HLS-Eval . . . 4
3.2 Analog macromodel . . . 5

4 Analog benchmarks: TRETS 2024 and JLPEA 2023 **5**
4.1 The artifact URL was already dead at publication . . . 5

4.2 Scaling (Figs. 3 and 11) . . . 5
4.3 Case II CAB arithmetic (Figs. 17–18) . . . 6
4.4 Executable Fig. 16, acoustic Case I . . . 6

5 A first project: AMS-Eval **7**

6 Papers read, not reproduced **8**

7 What I am not claiming **8**

1 What this note is

I read several of your papers with the aim of reproducing what can be reproduced from outside the lab, without proprietary EDA or an FPAA board, and saying plainly what cannot. Two threads are finished:

1. **HLS-Eval** [1] — a measurement paper. There is a pass@k table to match. Done 2026-08-12.
2. **TRETS 2024 analog benchmarks** [2] folded with **JLPEA 2023 analog HLS** [3] — a benchmark-definition paper plus its predecessor. There is no pass@k table. The reproducible claims are arithmetic (scaling curves, CAB costs) and making Fig. 16 executable. Done 2026-08-14.

A third thread is a *proposal*, not a result: **AMS-Eval**, the HLS-Eval metric ladder applied to analog block programs. SmartPAF and CRYPTONITE were read and are interesting; they were not run. Nothing below is a silicon measurement, and synthesis (Vitis / FPAA place-and-route) is out of scope throughout.

2 HLS-Eval: zero-shot generation, one year later

2.1 What was reproduced

The task is the paper's: given a natural-language kernel description, a header, and a testbench, generate C++. I score three of four staged metrics — parseable, compilable, runnable — at $n_{\text{samples}} = 5$, $T = 0.7$. Synthesizable is skipped, not failed: Vitis HLS is not installed. Every synthesis cell is recorded as skipped.

Scope. 57 designs from PolyBench, MachSuite, and C2HLSC, the three suites that compile and run at 100% under stock `clang++` with no Xilinx software. (A prior audit counted 60 such designs; the sweep used 57. That 3-design gap was not chased.) CHStone and Rosetta are excluded for missing testbenches and Xilinx-header reasons documented in `FINDINGS.md`.

Provenance. Upstream `hls-eval` @ `adea9ff` (2026-07-30), evaluated 2026-08-12, WSL2 Ubuntu, `clang++` 18.1.3, OpenRouter. The paper used Together AI. OpenRouter load-balances across providers and quantizations, so numeric precision is uncontrolled here in a way it was not for the paper. 1,140 LLM calls; 2 provider errors, both retried; 0 unrecovered.

The paper's other anchor, Qwen2.5-Coder-32B, was dropped before the sweep: every real HLS-Eval prompt to its sole OpenRouter provider (Cloudflare) returned `finish_reason: error`, while a short synthetic `curl` succeeded. DeepSeek-V3 is the sole paper anchor in this run.

2.2 Paper vs. this run

Model	parse@1	parse@5	compile@1	compile@5	run@1	run@5
DeepSeek V3 (paper, 94 dsn)	100.0%	100.0%	94.1%	97.6%	**63.3%**	**65.9%**
Qwen2.5-Coder-32B (paper)	95.5%	97.6%	90.8%	92.9%	57.6%	58.8%
deepseek-chat-v3-0324 (this run, 57 dsn)	63.2%	87.7%	61.4%	86.0%	46.3%	**63.2%**
95% CI	[54.0, 72.3]	[79.0, 94.7]	[52.3, 70.2]	[77.2, 94.7]	[35.8, 56.8]	[50.9, 75.4]

Table 1: Paper Table III (Together AI, 94 designs, Vitis included) vs. this run (OpenRouter, 57 designs, no synthesis). Bootstrap CIs over designs, 10,000 resamples.

The pass@1 gap is real; the pass@5 gap on runnable nearly vanishes (63.2% vs. 65.9%). A model that recovers by the fifth sample but fails the first is a budget or formatting problem, not a capability gap. `max_tokens=8192` is not in the paper's stated methodology and terminated 21% of this run's calls with `finish_reason=length`. Pass@1 numbers here are a **lower bound** on model capability.

Three confounds, stated plainly: (i) 57 designs vs. 94; (ii) OpenRouter vs. Together AI; (iii) the token cap.

2.3 Extension: models that postdate the paper

Model	parse@1	compile@1	run@1	run@5
deepseek-chat-v3-0324 (2025 anchor)	63.2%	61.4%	46.3%	63.2%
qwen3-235b-thinking-2507	73.7%	70.9%	52.3%	68.4%
deepseek-v4-pro	52.3%	49.5%	39.0%	61.4%
qwen3-coder	**99.7%**	**96.8%**	**67.4%**	**70.2%**

Table 2: Same harness and 57 designs. Qwen3-Coder is 21 points above the 2025 anchor at run@1. DeepSeek-v4-pro is weaker at pass@1 because it is the model hit hardest by the token cap (47.7% parse-failure rate), not because the newer checkpoint is less capable.

Qwen3-Coder saturates parse and compile, so the harness — not the model — is now the bottleneck at those rungs. One year of model progress did not translate uniformly under a fixed 8k token ceiling; the newer, more verbose models are more exposed to it.

2.4 What broke, independent of any model

Found by compiling the reference designs with a stock compiler, before any LLM call:

- 12 of 32 CHStone designs ship with no testbench. Runnable is undefined for them, not failed. Scoring them as failures understates CHStone runnability by 37.5%.
- `flowgnn/fgnn_linear`'s testbench calls `printf` without `<cstdio>`. Builds under Vitis's bundled clang; fails under GCC and modern Clang.
- `gnnbuilder/gather_node_neighbors`'s testbench writes out of bounds. GCC's stack protector aborts; Vitis clang does not enable it.

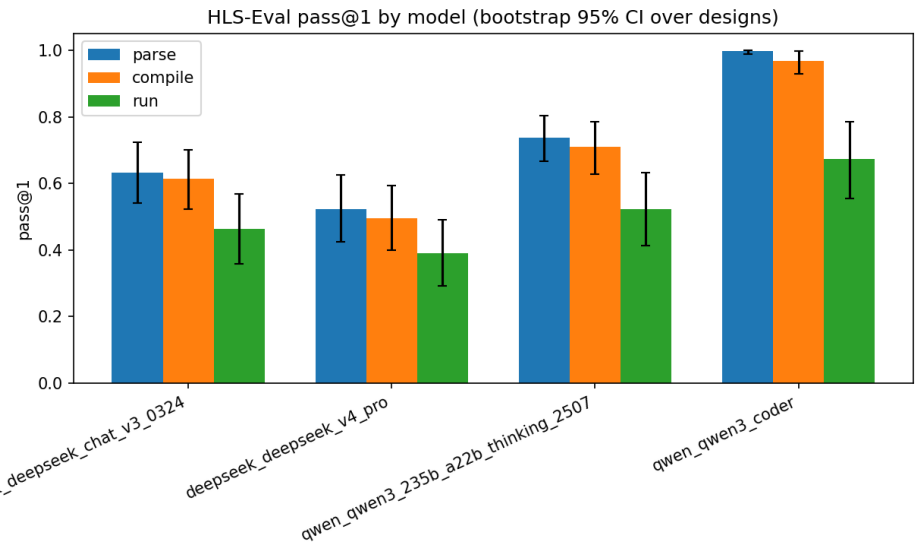


Figure 1: HLS-Eval pass@1 by model and metric, bootstrap 95% CIs over designs. Synthesis is omitted (no Vitis).

- The upstream harness has no execution timeout and inherits the parent cwd, so generated infinite loops hang a worker and relative-path testbench I/O reads the wrong directory. The Vitis-free backend used here sets both.
- The published leaderboard JSON in the upstream repo is still the dummy payload. This run emits a real payload in their schema.

3 Improvements I considered and did not run

The gaps above are specific enough that I could have spent another round of API credits (and a Vitis install) chasing a closer match to the published tables. I did not. The harness was frozen once the HLS sweep started, and the analog cut stopped at an executable Fig. 16 rather than at a silicon-matching accuracy. What follows is the improvement list I would actually run — in the group, or if someone re-runs these artifacts — not a claim that the current numbers are the best the models can do.

3.1 HLS-Eval

1. **Raise `max_tokens` to 12k–16k and re-sweep.** This is the single highest-leverage change. 21% of calls in this run ended with `finish_reason=length`; DeepSeek-v4-pro's 47.7% parse-failure rate and the 46% vs. 63% run@1 gap versus the paper are both consistent with truncation, not with a weaker model. A second $n = 5$ pass on the same 57 designs would likely lift pass@1 toward the paper and toward Qwen3-Coder's already-saturated parse rate. I left it undone so the reported numbers stay tied to one frozen harness, and so the token cap itself remains a finding rather than something I patched away.
2. **Re-test Qwen2.5-Coder-32B on Together AI**, the paper's actual provider. The Cloudflare/OpenRouter failure may be provider-specific. A Together key would tell us whether the

second paper anchor is reproducible at all. I did not add that key.

3. **Pin provider and quantization** (or disable OpenRouter load-balancing). The paper used one serving stack. This run did not. That confound sits underneath every numeric comparison in §2 even with an unlimited token budget.

4. **Install Vitis HLS** and restore synthesizable, the fourth metric. Three of four was a deliberate Vitis-free choice, not a claim that synthesis does not matter. The stock-compiler backend already showed that 60 reference designs need no Xilinx software for parse/compile/run.

5. **Bring CHStone and Rosetta back** once the 12 testbench-less CHStone kernels and Rosetta's Xilinx-header designs are handled, to reach the paper's 94-design scope. The 57-vs-60-design gap inside the already-clean suites should be closed in the same pass.

None of those change the qualitative result: Qwen3-Coder is ahead of the 2025 DeepSeek-V3 anchor under identical constraints, and the harness — not the student — is now the thing to fix first.

3.2 Analog macromodel

The 70% Case I number is a digital stand-in. The obvious lifts, not taken:

- Replace the integer LadderFM delay with a gm-C ladder from TRETS ref. [75] / the FPAA VM Xcos code.
- Train on the full Speech Commands v2 4-word split (this run used the mini subset) and, if licensed, compare against TIMIT/TI-digits as the paper does.
- Add the filter benchmark (ripple / stopband on a chirp) — fully specified in TRETS §3.4, no dataset, cheap — and Case II 32-word.
- Run the same Fig. 16 program on the lab's SoC FPAA, which is the only way 70% becomes a hardware number.

AMS-Eval (§5) is the improvement I actually want to build, not another accuracy chase on the macromodel.

4 Analog benchmarks: TRETS 2024 and JLPEA 2023

TRETS [2] defines analog/mixed-signal benchmarks (acoustic, vision, comm, filter) the way CHStone/MachSuite/Rosetta defined digital HLS ones. JLPEA [3] is the direct predecessor: same FPAA analog-HLS flow, with a worked acoustic classifier (subbanded C^4 BPF, ladder delays, VMM + winner-take-all). I folded them. There is no experimental table to match. What can be checked without an FPAA is (i) the scaling and CAB arithmetic, and (ii) running Fig. 16's level-1 program on a public dataset.

4.1 The artifact URL was already dead at publication

TRETS footnote 1: designs and numbers "openly available" at https://hasler.ece.gatech.edu/AnalogBenchmarks. Live check 2026-08-14: 404. Wayback CDX has two captures, both 2023-11-19 (six weeks after TRETS acceptance): HTTP 301, then HTTPS **404**. There is no archived suite. Authority for this reproduction is the PDFs (NSF PAR 10504737 and 10504736).

Afolabi Ige later open-sourced the *synthesis* tool (a-folabi/ashes). That is Python/XCOS → FPAA switch list / GDS, not the missing benchmark files. I did not clone it.

4.2 Scaling (Figs. 3 and 11)

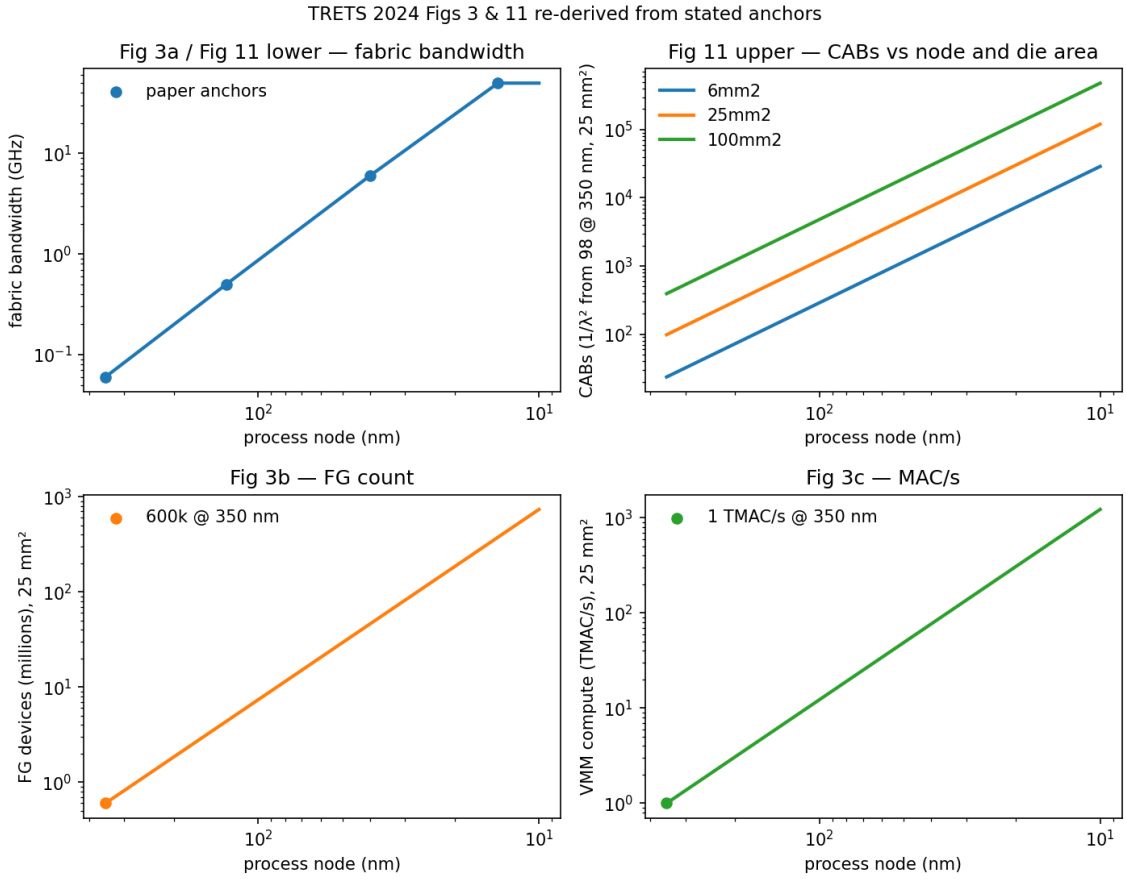


Figure 2: TRETS Figs. 3 and 11 re-derived from stated anchors, not from digitized pixels. Bandwidth interpolates the four caption points exactly. CAB/FG/MAC counts scale as $1/\lambda^2$ from the 350 nm SoC FPAA (600k FG parameters, ≈1 TMAC/s, 98 CABs on a 25 mm^2 reference die). If Fig. 11's drawn lines are shallower than λ^{-2}, that is a later overlay, not a fit.

Bandwidth anchors, by construction: 350 nm → 60 MHz, 130 nm → 500 MHz, 40 nm → 6 GHz, 14 nm → 50 GHz. Comm Case III (2 GHz on a 2 × 3 mm 40 nm FPAA [2]) is consistent with the 6 GHz 40 nm anchor — 3× margin. I did not re-read the 40 nm FPAA measurement paper.

4.3 Case II CAB arithmetic (Figs. 17–18)

The worked example is a 12-band AmplitudeBPF, three ladder delay stages → 48 lines, and a 48×128 VMM+WTA (32 words + 96 nulls) with a 6,144-weight matrix. Fig. 18 costs, SoC FPAA units:

Block	CABs	Rule
AmplitudeBPF × 12	12	Fig. 18: 1 CAB each
LadderFM, 12 ch × 3 stages	36	Fig. 18 LadderF rate (1 CAB/stage). LadderFM itself points at [5].
VMMWTA 48 → 128	128	1 WTA output / CAB on the SoC (body text)
WTA only	128	the sentence the paper actually writes
Full Fig. 17 program	176	

Table 3: Case II acoustic resource count. Weights $48 \times 128 = 6144$ match the paper exactly.

TRETS never reprints how many CABs the SoC FPAA has. Fig. 18 is "baselined to the CABs in the SoC FPAA [4]." That chip has **98 CABs**. Then $128/98 = \mathbf{1.31}$ chips — greater than one, less than two. The claim "more WTA stages than would fit on an SoC FPAA (barely)… two SoC FPAAs" **holds**, but only if that 98 is granted. If someone assumed ∼128 CABs on the SoC, WTA would be exactly one chip and "barely" would not hold. The full program is $176/98 = 1.80$ chips, still two. The 130 nm standard-cell area ($250\,\mu\text{m} \times 300\,\mu\text{m}$) was not re-derived.

Fig. 17 uses 12 bands. Body text elsewhere says "16 to 32 bands." I costed the figure.

4.4 Executable Fig. 16, acoustic Case I

JLPEA Fig. 3 is Case I: 16 subbands, ladder taps, VMM+WTA, 2–5 word vocabulary. TRETS Fig. 16 is the three-line program:

```
y1  = AmplitudeBPF(In, center=f1, Q=Q1, fpeak=f2)
y2  = M2V(LadderFM(input=y1, Ibias1=Ib1, Ibias2=Ib2))
Out = VMMWTA(input=y2, W=W1, Thresh=theta1, freq=100)
```

(Fig. 17's Case II listing writes `freq=400`; Case I follows Fig. 16.) I implemented those blocks as discrete-time macromodels (numpy/scipy; no ODEs, no FPAA VM). BPF is an RBJ biquad stand-in for the C^4; AmplitudeBPF is BPF + rectify + envelope LPF; LadderFM is an **integer sample delay** (25 ms/stage) — `Ibias1` and `Ibias2` are accepted and ignored. That is not a gm-C ladder. VMMWTA trains with a softmax surrogate and evaluates with hard WTA.

Dataset: TensorFlow mini Speech Commands, labels yes/no/up/down. TIMIT is LDC-licensed and was not used. 16 log-spaced bands, 50 Hz–4 kHz (Nyquist-limited relative to the paper's 10 kHz). Case I CAB cost is 68.9, which fits one 98-CAB SoC FPAA, consistent with "Case I already demonstrated on the SoC FPAA."

Held-out, $n = 200$ test / 600 train: **accuracy 0.70** (chance 0.25; normal-approx 95% interval $[0.64, 0.76]$). Per class: yes 0.78, no 0.57, up 0.89, down 0.53. This is not 90%, and it is not a silicon number. The paper's demonstrated classifier is analog, on TIMIT/TI-digits, with ladder dynamics this macromodel does not have. The deliverable is that Fig. 16 runs, not that the accuracy matches hardware.

Eight unit tests pass, including the 6,144-weight / 1.31-chip Case II arithmetic.

5 A first project: AMS-Eval

HLS-Eval's contribution is a metric ladder (parseable → compilable → runnable → synthesizable) plus a harness other people can re-run. TRETS asks for the analog analogue and then 404s the files. The obvious next step — and the one I would want to do in the group — is to reuse that harness shape on Fig. 15's block library:

HLS-Eval	AMS-Eval	Grade
parseable	parseable	AST; calls restricted to the Fig. 15 whitelist
compilable	simulatable	runs under the tier-F macromodel, bounded time
runnable	functional	accuracy / filter spec on held-out data
synthesizable	resource-feasible	`cab_cost(program)` $\leq$ Fig. 18 budget

The macromodels and CAB cost function from §4 are the grading backend. I did not run an LLM sweep on analog programs. If frontier models score near zero, that is itself the result ("they cannot compose analog block programs yet"). The six HLS-Eval harness fixes (timeout, cwd, pinned generation options, provider preflight, retry, larger `max_tokens`) would be baked in from day one rather than discovered mid-sweep.

This sits on the Hasler–Hao intersection: her digital HLS/benchmark methodology, his FPAA block library, an evaluation story the lab already knows how to read.

6 Papers read, not reproduced

SmartPAF (Tong, Dang, Golder, Raychowdhury, Hao, Krishna; MLSys 2024) [6] replaces ReLU/softmax/etc. with tuned low-degree polynomials for FHE inference, with claimed 1.42–13.64× accuracy and 6.79–14.9× speedup on ResNet-18 / ImageNet-1k. Public code was promised. This is the closest *numeric* analogue to HLS-Eval — standard model, standard dataset, concrete factors — but it is not analog/neuromorphic and ImageNet+FHE is a multi-day environment. I read it; I did not run it. Worth doing if the group cares about private inference, not as a second weekend project beside analog.

CRYPTONITE [7] auto-generates HLS accelerators for Fiat Cryptography primitives (88.88% resource reduction / 54.31% latency vs. naive synthesis) and is open-sourced. It wants Vitis HLS 2023.1 plus SVF/GiNaC/egg/LLVM. I went Vitis-free on HLS-Eval specifically to avoid that dependency. Interesting as an HLS compiler-stack paper; not AI/NN-flavored; not started.

TODAES survey of ML for software-assisted hardware verification [8] is background reading. Nothing to reproduce.

Two IEEE Xplore links from an earlier scan did not resolve (empty document ID; 418 on `11240893`).

7 What I am not claiming

- I did not match HLS-Eval's 94-design, 4-metric, Together AI setup. Three metrics, 57 designs, OpenRouter, token-capped.
- I did not run analog programs on an FPAA, and 70% is not a reproduction of a published accuracy. It is evidence that Fig. 16 has executable semantics.
- I did not digitize Fig. 11's pixels. CAB scaling is $1/\lambda^2$ from stated anchors.

- I did not implement AMS-Eval, SmartPAF, or CRYPTONITE.

The analog work is the same genre as the HLS-Eval work: run the benchmark definition as code, check the numbers that can be checked, write down what broke. The proposed first project is to put an LLM in front of the analog blocks the way HLS-Eval put one in front of C++ kernels.

Appendix: where the files are

- HLS-Eval: `hls-eval/` — report, `FINDINGS.md`, `results/leaderboard.json` (real payload, their schema), raw per-design output, Vitis-free backend in `ext/stock_tools.py`.
- Analog: `analog-benchmarks/ams-bench/` — `amsblocks/`, `analysis/scaling.py`, `examples/acoustic_cas` `FINDINGS_ams.md`.
- This PDF's figures: `report/figures/`.

References

[1] S. Abi-Karam and C. Hao, "HLS-Eval: A Benchmark and Framework for Evaluating LLMs on High-Level Synthesis Design Tasks," ICLAD 2025. https://arxiv.org/abs/2504.12268

[2] J. Hasler and C. Hao, "Programmable Analog System Benchmarks Leading to Efficient Analog Computation Synthesis," *ACM TRETS*, vol. 17, no. 1, Art. 12, Jan. 2024. https://doi.org/10.1145/3625298

[3] A. Ige, L. Yang, H. Yang, J. Hasler, and C. Hao, "Analog System High-Level Synthesis for Energy-Efficient Reconfigurable Computing," *J. Low Power Electron. Appl.*, vol. 13, no. 4, p. 58, 2023. https://www.mdpi.com/2079-9268/13/4/58

[4] J. Hasler, "A SoC FPAA with RISC microcontroller," *IEEE JETCAS*, 2016. TRETS ref. [31]. 98 CABs.

[5] Ladder-filter delay paper, TRETS ref. [75] / TCAS-I. Not re-implemented; LadderFM is an integer delay in this run.

[6] J. Tong, T. Dang, A. Golder, A. Raychowdhury, C. Hao, and T. Krishna, "Accurate Low-Degree Polynomial Approximation of Non-Polynomial Operators for Fast Private Inference in Homomorphic Encryption," MLSys 2024.

[7] K. Sharma et al., "CRYPTONITE: Scalable Accelerator Design for Cryptographic Primitives and Algorithms," arXiv:2505.14657. https://github.com/KarthikeyaSharma16/Cryptonite

[8] Wu, Li, et al. (incl. C. Hao), "A Survey of Machine Learning for Software-Assisted Hardware Design Verification," *ACM TODAES*, 10.1145/3661308.